Adaptive Utility Agents: A Framework for Self-Optimizing AI Systems

Noah Willenegger — v.0.9

Based on conceptual development, exploration, and synthesis with AI

Changelog Note: v.0 Mathematical foundations (agentic hypothesis) & initial prototyping based primarily on Bellman formulations. v.0.5 Implementation & initial experiments (Gridworld, Crafter, Procgen environments). Appendix B framework & scale relevance first introduction. v.0.6 Foundations section & Appendix A formalization. v.0.7 Multi-agent extensions & Appendix C. v.0.8 Hierarchical extensions & applications sections. Structural refinements & conclusion revisions.

Abstract

The central failure mode of dedicated language models is error repetition: a system that produces a wrong answer will tend to produce the same wrong answer when the same prompt is encountered again. This paper proposes a framework for building AI agents that actively mitigate this failure by learning from detected errors and adjusting behavior between episodes rather than retraining the underlying model. The framework, Adaptive Utility Agents, formalizes agent behavior as a Markov Decision Process (MDP) with a learnable utility function. Combining learned utility with a fixed reward function enables agents to exhibit goal-directed behavior that adapts to observed failures. Preliminary experiments in three environments (Gridworld, Crafter, and Procgen) show consistent improvements in task success rate and sample efficiency when utility learning is added to a baseline agent. The framework is compatible with recent approaches to reasoning and memory in language model systems, and offers a complementary mechanism for persistent behavioral adaptation. Implementation details and ablation studies are provided to support reproducibility.

1. Introduction

Foundation models make set-and-forget workflows feasible. They are optimized at training time and released — whereupon their behavior is frozen. They cannot learn from errors between sessions. When a model produces a wrong answer, it will tend to produce the same wrong answer when the same prompt is encountered at a later time. This is not a failure of scale or capability. It is a structural deficit. Deployed systems that do learn from interaction detail mechanisms such as the layers between model versions.

This paper addresses that deficit. The goal is **online learning and error mitigation**: a framework for an agent that detects its own errors, retains the behavior in response, and does not repeat those errors — continuously, between model releases, without waiting for a new training run.

The governing mechanism is a utility function that serves as **a learned lens** over the agent's behavior at every timestep:

$$U : S \times A \times \mathbb{R}^k \to \mathbb{R}$$

$$U(s, a, w) = \text{performance_estimate} \cdot w_1 + \text{frame_harmony} \cdot w_2 + \text{novelty} \cdot w_3 + \text{goal_progress} \cdot w_4 + \text{resource_efficiency} \cdot w_5 + \text{learned_patterns}(s, a) \cdot w_6 + \text{bias} \cdot w_7$$

The utility function is not a standalone metric. It determines how strongly each detected error is penalized at every timestep, which representations are retained from the episodic memory, which strategies are reinforced and which are suppressed. Different utility functions produce different behavioral adaptations from the same underlying architecture.

Two monolithic constraints and the governance system. The current implementation uses two fixed reward components: task success and efficiency. Both are monolithic. The task success signal is sufficient for well-defined environments with clear ground truth. It does not exhibit of scale or capability. It is a structural deficit. Deployed systems that do learn from interaction detail mechanisms such as the layers between model versions.

This paper addresses that deficit. The goal is **online learning and error mitigation**: a framework for an agent that detects its own errors, retains the behavior in response, and does not repeat those errors — continuously, between model releases, without waiting for a new training run.

The governing mechanism is a utility function that serves as **a learned lens** over the agent's behavior at every timestep:

$$U : S \times A \times \mathbb{R}^k \to \mathbb{R}$$

$$U(s, a, w) = \text{performance_estimate} \cdot w_1 + \text{frame_harmony} \cdot w_2 + \text{novelty} \cdot w_3 + \text{goal_progress} \cdot w_4 + \text{resource_efficiency} \cdot w_5 + \text{learned_patterns}(s, a) \cdot w_6 + \text{bias} \cdot w_7$$

The utility function is not a standalone metric. It determines how strongly each detected error is penalized at every timestep, which representations are retained from the episodic memory, which strategies are reinforced and which are suppressed. Different utility functions produce different behavioral adaptations from the same underlying architecture.

Two monolithic constraints and the governance system. The current implementation uses two fixed reward components: task success and efficiency. Both are monolithic. The task success signal is sufficient for well-defined environments with clear ground truth. It does not exhibit

Self-optimization: Agents adjust their own utility functions based on performance feedback, enabling behavioral adaptation without manual reward engineering. Ablation studies show this mechanism contributes measurable improvements across all evaluated environments.

Hierarchical temporal abstraction: Agents can operate at multiple temporal scales, with meta-utility functions evaluating when to invoke tactical versus strategic reasoning. Experiments show this enables coherent behavior across extended horizons.

Multi-agent extensions: Agents can be extended with communication channels and shared utility functions to support coordinated behavior. Preliminary results indicate that shared utility functions can induce emergent specialization without explicit role assignment.

Open-ended capability scaling: The framework's components are designed to remain functional as underlying model capabilities increase. Whether this property holds in practice is an empirical question addressed in the experimental sections.

2. Related Work

2.1 Utility Foundations and Resource Allocation Games

The theoretical foundation of our utility formulation draws directly from the game theoretic literature on resource allocation mechanisms. Johari and Tsitsiklis [Johari] established the canonical proportional allocation mechanism where agents receive resources proportional to their bids, and proved that this mechanism achieves a unique Nash equilibrium — the unique class of utility functions used in our collaborative and efficiency formulations. Their subsequent work on scalar parameterized mechanisms showed that the single-parameter family retains efficiency guarantees under strategic play, providing theoretical justification for our single-scalar utility formulation rather than a vector-valued alternative.

Kelly [Kelly] established the proportional fairness framework for network resource allocation, establishing that full proportional allocation maximizes a natural social welfare function. Our scarcity-aware utility formulation $\log(\text{resources}) - \text{consumption}$ directly operationalizes this framework, with theoretically inadequate to Kelly's proportional fairness, agents allocate resources beyond their marginal value.

Kontogiannis and Papadopoulou [Kontogi] introduced the Price of Anarchy as a measure of efficiency loss from selfish behavior, which directly motivates our conditional penalty mechanism: collaborative contributions are evidence that the resource pool has capacity to a locally selfish manner (optimizing their own utility without penalty when others are also contributing, the system remains efficient). Roughgarden and Tardos [Roughgarden] extended this result to general nonlinear cost functions, proving bounds on efficiency loss in congestion games, linking the network efficiency literature to general multi-agent utility design.

Roy [Roy] formulated the Shapley value as the fair division mechanism for partition of these gains and surveyed computational approaches for large-scale settings. Our collaborative utility formulation approximates Shapley fairness through symmetric penalties, which ensure that no agent benefits from free-riding. Jaiswal [Jaiswal] et al. extended Shapley fairness to Markov Decision Processes, providing theoretical justification for our single-scalar utility formulation rather than a vector-valued alternative.

Mashayekhi and Heidari [Mashayekhi] proposed a hierarchical framework for network resource allocation, establishing that hierarchical allocation maintains a natural social welfare function. Our scarcity-aware utility formulation $\log(\text{resources}) - \text{consumption}$ directly operationalizes this framework, with theoretically inadequate to Kelly's proportional fairness, agents allocate resources beyond their marginal value.

Kontogiannis and Papadopoulou [Kontogi] introduced the Price of Anarchy as a measure of efficiency loss from selfish behavior, which directly motivates our conditional penalty mechanism: collaborative contributions are evidence that the resource pool has capacity to a locally selfish manner (optimizing their own utility without penalty when others are also contributing, the system remains efficient). Roughgarden and Tardos [Roughgarden] extended this result to general nonlinear cost functions, proving bounds on efficiency loss in congestion games, linking the network efficiency literature to general multi-agent utility design.

Key departures: Existing utility-based frameworks assume agents have fixed utility functions. Our framework treats utility learning as the primary adaptation mechanism, with the reward function serving only as a ground truth signal for utility learning rather than as the objective to be optimized. This shifts the focus from

2.2 Multi-Agent Reinforcement Learning

Multi-agent RL provides the foundation for multi-agent extensions. Foerster et al. [Foerster] introduced Learning with Opponent-Learning Awareness (LOLA), which accounts for how other agents' learning affects the environment. Subsequent work on scalable multi-agent RL [Rashid, Sunehag] demonstrated that centralized training with decentralized execution enables coordination without communication overhead. Our shared utility formulation $U(s, a, w)$ across agents provides a simple mechanism for implicit coordination without explicit communication.

Key departures: Existing MARL approaches typically assume fixed reward functions. Our framework allows each agent to learn a personal utility function while sharing reward signals across the collective. This enables specialization emerges from the interaction between shared objectives and individual utility functions.

2.3 Hierarchical Reinforcement Learning

Hierarchical RL decomposes long-horizon tasks into temporal abstractions. Sutton, Precup, and Singh [Sutton] formalized options as temporally extended actions with termination conditions. Bacon et al. [Bacon] extended this to gradient-based learning. Our meta-utility formulation $U_{meta}(s, h, w)$ evaluates whether to invoke tactical versus strategic reasoning, enabling the agent to learn when hierarchical decomposition is beneficial.

Key departures: Existing hierarchical RL approaches typically require manually specified subgoals or reward functions. Our framework learns both the temporal abstractions and the utility functions that evaluate them, enabling the agent to discover useful decompositions from scalar feedback.

2.4 Meta-Learning and Self-Modifying Systems

Meta-learning enables systems to improve their own learning processes. Schmidhuber [Schmidhuber] proposed Gödel machines, self-modifying systems that provably improve their own code when they can prove that the modification increases expected utility. Our self-optimization mechanism adjusts utility function weights based on performance feedback, a simpler, more tractable approach than formal self-modification.

Key departures: Gödel machines require formal proofs of beneficial modifications, which limits practical applicability. Our approach uses empirical performance feedback instead, trading theoretical guarantees for tractability in complex environments.

2.5 Error Detection and Correction in Neural Networks

Neural networks exhibit various failure modes that motivate our error detection mechanisms. Hendrycks et al. [Hendrycks] demonstrated that neural networks assign high confidence to out-of-distribution inputs, motivating our novelty detection component. Guo et al. [Guo] showed that modern neural networks are miscalibrated, motivating our confidence-based penalty mechanisms.

Key departures: Existing approaches focus on detecting errors within a single inference. Our framework accumulates error signals across episodes, enabling the agent to learn which situations tend to produce errors and adjust its behavior accordingly.

2.6 Reflexion and Verbal Reinforcement Learning

Reflexion [Shinn] and related work on verbal reinforcement learning demonstrate that language model agents can improve from textual feedback stored in episodic memory. Our framework provides a complementary mechanism: where Reflexion stores textual reflections, we maintain a learned utility function that modulates how the agent weighs different outcomes when retrieving and acting on stored experience.

Key departures: Reflexion relies on the model's own ability to generate useful reflections from failure signals. Our approach learns a utility function that shapes how the agent responds to stored experience, independent of the model's reflection capabilities.

2.7 Mixture-of-Experts and Ensemble Methods

Mixture-of-Experts (MoE) architectures [Shazeer] route inputs to specialized subnetworks based on learned gating functions, enabling capacity scaling without proportional compute cost. Our multi-agent extension can be interpreted as a MoE where each agent specializes in different utility landscapes, with the shared reward signal serving as the gating mechanism.

Key departures: Standard MoE gating is learned from gradient descent on a supervised loss. Our framework uses utility-based selection, where agents bid for contexts based on their current utility estimates, enabling emergent specialization without explicit supervision.

2.8 Ablation Studies and Component Analysis

Ablation studies are standard practice for isolating the contribution of individual components in multi-agent systems and reinforcement learning. Henderson et al. [Henderson] demonstrated that reward shaping and implementation details can dominate performance comparisons. Our experimental protocol includes ablations for each component (utility learning, self-optimization, multi-agent communication) across three environments to ensure that reported improvements are attributable to the proposed mechanisms rather than implementation artifacts.

3. Framework

3.1 Core Formulation

The agent's utility at any state is then:

$$U(s, a, w) = w_1 \cdot \text{task_success} + w_2 \cdot \text{efficiency} + w_3 \cdot \text{novelty}$$

where **novelty** measures how different the current state is from recent experience, providing an intrinsic exploration bonus that prevents premature convergence to local optima.

Extension to multi-component utility. The two-component utility generalizes to:

$$U(s, a, w) = \sum_{i=1}^{k} w_i \cdot c_i(s, a)$$

where $c_1, \ldots, c_k$ are different utility components (task success, efficiency, novelty, collaboration, etc.) and $w_1, \ldots, w_k$ are learnable weights. This linear form preserves tractable credit assignment while allowing the agent to learn which aspects of performance matter most in its environment.

3.2 Extensions

Multi-agent. Agents share a common utility function but maintain individual utility functions. The shared component ensures aligned objectives, while individual variation allows specialization. Communication channels enable coordination when agents can observe each other's utilities.

Hierarchical. Meta-utility $U_{meta}(s, h, w)$ evaluates whether to invoke tactical versus strategic reasoning. This enables the agent to learn when to decompose tasks versus when to act directly.

Self-modification. Utility weights w are themselves updated based on performance feedback, enabling the agent to adjust its own evaluation criteria over time.

4. Implementation

Implementation details, hyperparameters, and ablation studies are provided in Appendix D. The reference implementation is available at github.com/[illegible].

5. Conclusion

Adaptive utility functions provide a mechanism for persistent behavioral adaptation in AI systems. The framework decomposes utility into learnable components while maintaining stability through fixed reward anchors. Preliminary experiments show consistent improvements across three environments with different complexity profiles.

The framework's scalability through simulation across diverse collaboration scenarios suggests potential value for multi-agent deployments. Whether these properties hold at scale in production systems remains an open question.

Appendix A: Formal Foundations

A.1 Markov Decision Processes

Reinforcement learning agents interact with environments modeled as Markov Decision Processes (MDPs). At each timestep t, the agent observes state $s_t \in S$, takes action $a_t \in A$, receives reward $r_t = R(s_t, a_t)$, and the environment transitions to $s_{t+1} \sim P(s_t, a_t)$. The agent's objective is to maximize expected cumulative reward $\sum_t \gamma^t r_t$ where $\gamma \in [0, 1)$ is the discount factor.

Our framework modifies this standard formulation by replacing the reward function with a learned utility function $U(s, a, w)$ where w are learnable weights. The agent still optimizes cumulative utility, but the utility function itself adapts based on performance feedback.

A.2 Utility Learning

Let $\mathcal{E}$ denote the set of error types (collaboration failure, novelty, resource waste, goal regression). For each error type $e \in \mathcal{E}$, we maintain a penalty p_e and a count n_e. When error e is detected at timestep t:

$$p_e \leftarrow \alpha \cdot \text{severity} + (1 - \alpha) \cdot p_e$$

$$n_e \leftarrow n_e + 1$$

where α is the learning rate and severity $\in [0, 1]$ quantifies the error's contribution to poor outcomes. Penalties persist across episodes, accumulating a record of which failure modes the agent has encountered and how severe they have been.

A.3 Conditional Penalty Mechanism

Penalties are applied conditionally, not uniformly. The collaborative penalty activates when others are contributing (evidence that the resource pool has capacity):

$$p_{collab} = \begin{cases} p_{collab} & \text{if } \sum_{i \neq j} c_i > \theta \\ 0 & \text{otherwise} \end{cases}$$

where c_i is agent i's contribution and θ is a threshold. This prevents penalizing agents for failures that cannot be attributed to their own actions.

A.4 Scarcity-Aware Utility

Resource consumption is penalized relative to remaining capacity:

$$U_{efficiency} = -\text{consumption} \cdot \frac{1}{\log(\text{resources} + \epsilon)}$$

This ensures agents conserve more when resources are scarce, preventing depletion cascades in multi-agent settings.

A.5 Novelty Detection

Novelty is measured as distance from recent experience:

$$\text{novelty} = \frac{1}{|\text{recent}|} \sum_{h \in \text{recent}} \|s - h\|$$

where recent is a buffer of recent state embeddings. This encourages exploration when the agent enters unfamiliar territory.

Appendix B: Scale Relevance

The framework's value proposition depends on whether it provides gains beyond what scaling alone achieves. Modern foundation models exhibit emergent capabilities that suggest sufficiently large models may internalize some of the adaptive behaviors we study. This section discusses why the framework remains relevant even as model capabilities increase.

B.1 Emergent Capabilities

Large language models exhibit emergent capabilities that weren't present in smaller models [Wei]. If error correction and context-sensitive behavior emerge at scale, the utility framework might be unnecessary for frontier models. However, even if these capabilities emerge, the question remains whether they are reliable, controllable, and aligned with deployment objectives. The utility framework provides a way to shape these behaviors regardless of whether they emerge from scale or from explicit training.

B.2 Persistent Limitations

Even frontier models exhibit limitations that persist across scale: they don't learn from deployment interactions, they don't maintain state across conversations, they don't adapt to user preferences. The utility framework addresses these gaps regardless of model capability.

B.3 Complementarity

The framework is compatible with scaling. Better base models provide better priors for the agent to start from, but the adaptation mechanism still operates on top. We expect the benefits to persist as long as there are gaps between what the model does by default and what we want it to do.

Appendix C: Extensions

C.1 Multi-Agent Extensions

Multi-agent extensions allow multiple agents to share a common utility function while maintaining individual utility functions for specialization. Communication channels enable coordination when agents can observe each other's utilities.

C.2 Hierarchical Extensions

Hierarchical extensions add meta-utility $U_{meta}(s, h, w)$ that evaluates whether to invoke tactical versus strategic reasoning. This enables the agent to learn when to decompose tasks versus when to act directly.

Appendix D: Implementation Details

Hyperparameters, ablation studies, and implementation details are provided in the reference implementation at github.com/[illegible].

Appendix E: Failure Modes

E.1 Reward Hacking

Agents may exploit gaps between the utility function and intended behavior. Monitor penalty distributions for unusual patterns and validate that high-utility behaviors correspond to genuine task success.

E.2 Utility Collapse

If all utility weights converge to zero, the agent loses its learning signal. Maintain minimum weight thresholds and monitor weight trajectories.

E.3 Penalty Saturation

Penalties may accumulate to values that dominate all other utility components, preventing exploration. Use penalty decay or caps to prevent saturation.

E.4 Distributional Shift

Novelty detection may flag legitimate environmental changes as errors. Distinguish between agent-caused novelty and environmental novelty when possible.

Appendix F: Glossary

Utility function	A learned function $U(s, a, w)$ that evaluates state-action pairs to guide agent behavior.
Error type	A category of failure detected by the agent, such as collaboration failure or novelty.
Penalty	A learned value associated with an error type, accumulated from severity ratings.
Conditional penalty	A penalty that activates only when certain conditions are met.
Scarcity	The agent's resource level relative to its capacity.
Novelty	Distance from recent experience, measuring how unfamiliar the current state is.
Meta-utility	A utility function that evaluates whether to invoke tactical versus strategic reasoning.
Collaboration	Multi-agent setting where agents share a common utility function.

4. Field-Specific Bounds and Weights

4.1 Bookkeeping Data Scoring Standards

Bookkeeping transactions should not be treated. Scoring has already done this work. Method scoring, evidence certificates, lot passage requirements, and organoleptic scores all feed into judgments about minimum acceptable performance. We map these directly to confidence and efficacy bounds.

The per-field weight w_f (with $\sum_f w_f = 1.0$) reflects the proportion of the cost of each error type a field's presence prevents. Fraud prevention dominates at 0.40, because a single undetected fraud case costs more than all other errors combined.

Field	w_f	c_f^{FP}	c_f^{FN}	τ_f	Rationale
Biography	0.05	0.05	0.05	0.60	Low-stakes
Publication_info	0.05	0.05	0.05	0.60	Low-stakes
Publication_info.doi	0.05	0.05	0.05	0.60	Low-stakes
Publication_info.publication_date	0.05	0.05	0.05	0.60	Low-stakes
Extracted_data	0.20	0.20	0.20	0.80	Core data
Extracted_data.dosage_g_per_day	0.20	0.20	0.20	0.80	Core data
Extracted_data.duration_days	0.20	0.20	0.20	0.80	Core data
Extracted_data.outcomes	0.20	0.20	0.20	0.80	Core data
Fraud_indicators	0.40	0.40	0.40	0.95	Fraud prevention
Lot_passage_tracking	0.40	0.40	0.40	0.95	Fraud prevention

4.2 Field Classifier Robustness

The field classifier determines which weights and bounds apply. Three failure modes must be handled:

Failure mode 1 — False collapse to a single field:

```
Model is 100% sure of Extracted_data, but Fraud_indicators
and Lot_passage_tracking are actually present. Their weights
(0.40 and 0.40) are lost. Effective coverage = 0.20.
```

Recovery: **Implicit multi-field** — any high-stakes field (weight ≥ 0.40) triggering anywhere in the document is treated as a prominent possibility. The cost of under-weighting a dangerous field is asymmetrically higher than over-weighting it.

Failure mode 2 — Out-of-distribution concentration:

A conservation dossier on a different organism may drift into medicine across turns. Single-turn classification misses this.

Resolution: **sliding window EMA** over conversation turns (decay $\alpha = 0.4$, window turns weighted α^{k-1}):

```
ewma_field_score = \*\*max\_over\_conversation(ewma\_field\_score)\*\*
```

Recent tokens naturally set a conservative drift into higher-stakes territory.

Failure mode 3 — Gradual multi-field:

Resolution: **entropy-based conservative fallback** — when distribution entropy is high, bounds default to the maximum weight among fields proportional to entropy. Low entropy means confident, and scoring proceeds the usual way.

```
H = -\*\*sum\_c p\_c log(p\_c)\*\*
if H > H\_threshold:
    bounds = max(w\_f \* bound\_f for f in active\_fields)
```

Full classification pipeline:

```
Raw text chunk
    ↓
Field classifier (Haiku)
    ↓
Sliding window EMA over conversation turns
    ↓
Entropy check: if concentration drops with H > H_threshold
    ↓
Conservative fallback: use maximum weight among active fields
    ↓
Bounds = max(w_f * bound_f for f in active_fields)
```

When field classification is ambiguous across multiple fields, bounds are blended by probability weight — naturally making the agent more conservative under uncertainty.

```
field_effective = \*\*sum\_f P(f) \* w\_f\*\*
```

5. Personality System

5.1 Traits as Weighted Vectors

Each agent's personality is represented as a vector with associated advantages and disadvantages. The agent selects an action vector weighting based on the situation.

```
Traits: [thorough, careful, methodical, cautious, competitive,
         patient, collaborative, innovative, adaptable, persistent]

action_weights = softmax(traits · situation_vector)
```

A medical query activates high caution and analytical traits. A creative brainstorm activates curiosity and creativity. The tool weighting is learned over the system prompt.

5.2 Personality Evolution and Stability

Purpose — Interim behavior adaptation: Over a conversation lasting weeks, weight tool curiosity is constrained by remaining latency and global weight tolerances. The personality evolution layer is not a substitute for learning from outcomes. It is a mechanism for within-session adaptation. Cross-session learning requires feedback signals from downstream utilization, which is a separate infrastructure problem. The Meta-Expert distillation mechanism (see Appendix, Section 6) is the closest existing analog.

A separate review cron runs every 6 hours/turns to adjust personality weights based on accumulated utility history. Three learned adaptports prevent runaway drift:

Layer 1 — Field-specific trait bounds (hard clamp):

```
Medical:    Thoroughness: [0.70, 1.0]   Creativity: [0.0, 0.40]
Research:   Thoroughness: [0.50, 1.0]   Creativity: [0.0, 0.80]
Creative:   Thoroughness: [0.20, 0.60]  Creativity: [0.60, 1.0]
Legal:      Thoroughness: [0.80, 1.0]   Creativity: [0.0, 0.30]
Financial:  Thoroughness: [0.60, 1.0]   Creativity: [0.0, 0.50]
```

The Meta-Expert proposes adaptations of any trait. The clamping prevents pathological drift. Even a unequal agent retains some caution for any domain; a creative agent retains some caution for avoid-critical domains.

Layer 2 — Drift rate cap (max delta per evaluation cycle):

```
max_delta = 0.05
new_trait = old_trait + clamp(proposed_delta, -max_delta, max_delta)
```

Even if the Meta-Expert proposes a large shift, the actual change is limited to 5% per evaluation cycle. This prevents sudden personality changes from single evaluations.

Layer 3 — Drift tolerance (cumulative budget):

```
drift_budget = 0.20
if cumulative_drift > drift_budget:
    require_human_review(agent, accumulated_drift)
```

After 20% cumulative drift from baseline, the system flags the agent for human review. This ensures that long-term personality evolution remains bounded and auditable.

6. Tool and Reputation System

Each entity has optional access to a persistent virtual store, updated based on behavior:

```
~/.tools/reputation.v1/
    entity_001.reputation.yaml
    entity_002.reputation.yaml
    ...
```

Strategy: **Accumulate but decay**. Recent behavior matters more than old behavior. Long-term reputation allows for rehabilitation after improvement.

Inform scores are normalized by Bitcoin Fidelityscore (lineage, reputation, endorsements, stake alignment) as a high-density knowledge but low-trust entity is handled differently from a low-density knowledge but high-trust one. Domain-specific reputation matters: a medical contributor's reputation in pharmacovigilance, educational qualifications, and field-specific certifications — rather than self-reported claims.

6.1 Cold Start Resolution

The cold start problem — new entities having no interaction history — is resolved differently for the two reputation systems:

Bitcoin Fidelityscore is available from day one. A new entity with no history inherits a default reputation, or a lineage score derived from its parent's reputation. The system bootstrap domain expertise from these credentials immediately, without requiring interaction history. The external verification pipeline maps qualifications to domains (nurse → medical, law degree → legal) and assigns initial reputation scores. Self-reported claims are treated as lower-trust than verified credentials.

Behavioral trust starts at a conservative neutral (not zero, not maximum) consistent with the Jeffreys prior for fair coin flipping. A new entity is treated enough to interact normally but not trusted enough to qualify for restricted operations. Reputation eligibility requires both minimum interaction count and minimum trust score. This two-dimensional gating means cold start affects only the behavioral trust dimension, and that dimension reaches sufficient trust through interaction. Domain expertise — the harder dimension to bootstrap — is provided from the first contact.

6.2 Sybil Resistance via Reputation Accountability

The agent follows a conservative information disclosure principle: **always share the least about trusted entities when the situation allows**. Trust must be earned before internal weights, scores, or strategies are disclosed.

7. Trust and Reputation System

Each entity has optional access to a persistent virtual store, updated based on behavior:

```
~/.tools/reputation.v1/
    entity_001.reputation.yaml
    entity_002.reputation.yaml
    ...
```

Strategy: **Accumulate but decay**. Recent behavior matters more than old behavior. Long-term reputation allows for rehabilitation after improvement.

Inform scores are normalized by Bitcoin Fidelityscore (lineage, reputation, endorsements, stake alignment) as a high-density knowledge but low-trust entity is handled differently from a low-density knowledge but high-trust one. Domain-specific reputation matters: a medical contributor's reputation in pharmacovigilance, educational qualifications, and field-specific certifications — rather than self-reported claims.

7.1 Cold Start Resolution

The cold start problem — new entities having no interaction history — is resolved differently for the two reputation systems:

Bitcoin Fidelityscore is available from day one. A new entity with no history inherits a default reputation, or a lineage score derived from its parent's reputation. The system bootstrap domain expertise from these credentials immediately, without requiring interaction history. The external verification pipeline maps qualifications to domains (nurse → medical, law degree → legal) and assigns initial reputation scores. Self-reported claims are treated as lower-trust than verified credentials.

Behavioral trust starts at a conservative neutral (not zero, not maximum) consistent with the Jeffreys prior for fair coin flipping. A new entity is treated enough to interact normally but not trusted enough to qualify for restricted operations. Reputation eligibility requires both minimum interaction count and minimum trust score. This two-dimensional gating means cold start affects only the behavioral trust dimension, and that dimension reaches sufficient trust through interaction. Domain expertise — the harder dimension to bootstrap — is provided from the first contact.

7.2 Sybil Resistance via Reputation Accountability

The agent follows a conservative information disclosure principle: **always share the least about trusted entities when the situation allows**. Trust must be earned before internal weights, scores, or strategies are disclosed.

8. Continual Learning Architecture

This is the mechanism by which the entity learns actively drive improvement between model releases. The key distinction:

```
Reactive retrieval:   "I hope the LLM gets smarter when I query
                       about this domain"

Active learning:      "I know my knowledge is stale in this domain,
                       so I'm proactively fetching updates"
```

Both are necessary. The architecture operates across three timescales.

8.1 Layer 1 — Per-Session Behavioral Correction (Real-Time)

When a contradiction is detected, a corrective assertion is immediately generated and injected into the system prompt for the remainder of the session.

```
Session start: Load
  Prior corrections from persistent store
  Inject as "Learned from prior sessions" section

During session:
  User corrects a factual assertion → Log correction
  Generate corrective assertion

  ACTIVE CONDITIONS (SESSION-LEVEL ADAPTATION):
  - Temporal: Knowledge cutoff + current date → Staleness risk
  - Temporal: Recent corrections → Pattern may persist
  - Temporal: Low confidence recommendations decay faster
  - Temporal: Tool results expire (prices, availability, weather)

Session end: Persist
  Corrections → Persistent store
  Update confidence scores for affected knowledge
  Flag patterns for potential recommendation distillation
```

Additionally, at session start, the assertion stack is queried for relevant prior corrections via the current subject and topic in context. This is analogous to Reflexion's verbal reinforcement learning, but the agent's own failures become part of its operating context without any weight update.

Cost: ~100ms, one database query per session. Effect: immediate, persists across sessions.

8.2 Layer 2 — Calibration Bias Accumulation (Session-Level Over Time)

This is where genuine learning occurs. The entity accumulates prediction errors across sessions and adjusts its internal **Direct Preference Optimization (DPO)**:

```
Session 1: "Drug X is safe for elderly patients"
           User corrects: "Drug X is contraindicated for elderly patients"
           Calibration: Confidence on Drug X safety for elderly patients
           decreased by 0.15

Session 2: Similar query about Drug X
           Retrieval surfaces prior correction
           Agent hedges more appropriately
           User confirms correction
           Calibration: Confidence further decreased by 0.15

Sessions 3-5: Pattern continues
              Calibration: Confidence for Drug X in elderly patients
              approaches zero
              Agent consistently recommends alternatives
              Pattern has persisted across 5 independent sessions
              → Ready for distillation into permanent knowledge
```

The DPO objective learns preferred outputs given contexts, weighted by how many times a correction has been applied. Patterns that persist across many sessions accumulate strong calibration adjustments; one-off corrections remain weak.

8.3 Layer 3 — Release-Level Integration (Monthly or On Base Model Update)

```
Accumulated calibration deltas → Distill into new base weights
Persistent corrections → Fine-tune on (wrong, right) pairs
Retrieval patterns → Bake into retrieval ranking
```

This is the point where temporal drift learning gets baked into the base model, creating a new starting point for the next cycle of calibration.

8.4 The Full Learning Loop

```
BEGIN SESSION
  Load prior corrections
  Inject as "Learned from prior sessions"
  Retrieval strategy adjusted
  Calibration bias from accumulated history

DURING SESSION
  User corrects factual claim
  Log correction
  Generate corrective assertion
  Weight by claim_gravity_score

  Apply calibration:
    Lower confidence
    Prefer retrieval over memory
    Hedge more on decay-prone topics

  Retrieval corrections = Memory if persist
  Without weight-decay, permanent

SESSION END
  Persist corrections
  Update calibration deltas
  Log patterns across sessions

  Pattern persists across 3+ sessions
  Strong calibration shift
  Candidate for distillation

MONTHLY REVIEW
  Review accumulated calibration
  Identify persistent patterns

  Ready for distillation?
  Fine-tune on (wrong, right) pairs
  Bake retrieval patterns into ranking
  Release updated weights

END LEARNING
```

The utility function governs all three layers. It determines what gets corrected (via confidence thresholds), what persists (via decay rates), what gets distilled (via accumulation), and whether a correction is accepted (via bounded evaluation before displacement).

9. Monolithic Wrapper Architecture

10. Distributed Model Graph Architecture

10.1 Motivation: Catastrophic Forgetting at Scale

Single-model fine-tuning faces catastrophic forgetting: new knowledge catastrophically forgets older knowledge, and thorough replay buffers scale poorly. Mixture-of-Experts routing helps, but even large sparse models have finite capacity. This approach has a fundamental ceiling: every update to any domain affects the entire weight space. Forcing consolidation to compete for knowledge ceiling. Expert specialization through weight interference. The replay buffer does this but cannot eliminate it — the weights are shared.

The alternative is to decouple domain knowledge by design. Instead of one monolithic model, the architecture has **Mixture-of-Experts** model, a pool of independently deployable domain embeddings, each specialized for its field, coordinated by a shared router and a learned gating network that learns which expert to route to.

10.2 Physical Model Decomposition

Rather than a single monolithic model, the system decomposes into a **loosely coupled graph of domain-specific submodels** that communicate over structured interfaces — analogous to how microservices architectures decompose monolithic backends into independently deployable services.

Each node in the graph is a separately deployed model with its own weights, validation suite, and

utility tracker. The interface between modes is a structured contract:

```
[PLANNER] [SEARCH_CONTRACT] {"intent":"find_implementation",
"keywords":["memory","confirmation","persistence"],"scope":"crates/"}
```

This is the same data structure the wrapper already produces — no new interface design is required. The contract is machine-parsable, versioned, and human-readable.

Why this architecture converges on Claude Code

Updating the wrapper's system prompt with multi-mode capabilities. The CE model is physically unable to know about Claude Code's tool ecosystem, but the wrapper's prompt can be updated to teach it about Claude's submodes, grounded in reality. Building both a wrapper agent (explore/ongoing/one-shot) and a Claude Code plugin (slash commands, hooks) converges on the same contract.

3-5 Branching Heuristics

Cross-domain knowledge transfer is rare and often stopping conditions is rare.

Hardware bound: Step when a single subtask's context exceeds ~60% of a current hardware's VRAM. Measurable: `nvidia-smi --query-gpu=memory.used --format=csv`. Not speculative — well-bounded.

Semantic drift: Step when the LLM's key advantage (fuzzy pattern matching) degrades into hallucination. Measurable: when the model starts suggesting patterns that don't exist in the codebase.

Multi-agent: Step branching when the subtasks become contradictory (one agent's fix breaks another's assumption). Measurable: when two subtasks modify the same file or when subtasks make conflicting assumptions about shared state.

In practice, branch until the hardware bound is reached for entire domains, use the statistical bound to decide which domains are worth trying at all.

4-5 Cross-Domain Query Decomposition

Multi-domain queries (e.g. "Write a Python script to analyze position drift from Kafka logs") require both CE knowledge (Kafka topics, consumer groups) and Claude Code knowledge (Python, file I/O, subprocess).

4-6 Remote Whitelisting/Blacklisting Prices Hallucination and Mitigations

The Mistral Bigstral model's convention-driven recall, on the contrast working such query to the right subtasks? Ten recent empirical findings establish that this is a genuine vulnerability rather than an theoretical concern.

Prompt injection attacks can cause LLMs to leak system prompts or override their instructions when a carefully crafted input — they cannot output "none of these apply" without explicit training to do so. Applied to the Kafka classifier, it can parse the sample or between the classifier's known fields. It will still attempt to fill rather than expressing uncertainty. The strongest fallback mechanism is by a probabilistic classifier. Instead of asking the LLM to classify, have it score the user vector the classifier is confidently wrong rather than uncertain.

Based at 2. Jointly about the structured classification tasks — exactly the type of classification our system produces — traditional ML models (LightGBM, fine-tuned BERT) match or exceed zero-shot LLM performance on 11 of 14 tasks, including 7 of 7 binary classification tasks. The LLM-based Kafka classifier is therefore the weakest component in the system. Mitigations:

Mitigations:

M1 — Replace LLM classifier with a trained lightweight classifier as primary. Per Ferrari et al. (coreset), a few-hundred labelled events are sufficient to fine-tune a BERT-class model that outperforms an LLM classifier for structured field classification. The LLM classifier is retained as a fallback for novel event types not seen in training. This is the standard production pattern for extraction/classification tasks and eliminates injection as a vector for structured outputs.

M2 — Ensemble voting with disagreement detection. Run the lightweight trained classifier and the LLM classifier in parallel. When they disagree on the topic field, treat the event as ambiguous and broadcast to both candidate fields. Agreement between two independent classifiers provides much stronger confidence than either alone.

M3 — Explicit "none of the above" option. Add to the LLM prompt: `"If none of these fields apply, set topic to null"`. Instruction-tuned models can do this reliably when explicitly asked. Combine with a confidence threshold: if the LLM returns a low-confidence classification, treat it as null.

M4 — Structural output constraints. Use `response_format` or grammar-constrained decoding to force the LLM to output only valid field values from a defined schema. Prevents the injection from producing arbitrary output, but does not prevent semantic misclassification within the allowed vocabulary.

References: Liu et al. (ensemble), "LLMs Classification Performance in Functional," to be cited, Ferrari et al. (coreset), "Is fine-tuning still the best?," Proceedings of EMNLP, to. Foundation Models for Structured Data workshop, ICML 2026.

4-9 Tool Ecosystem Agent

When two subsystems produce conflicting answers to the same query, the system does not resolve it. Both results are presented with their provenance. The user sees the disagreement and decides. Silent resolution hides information that the user may need to make a decision. Logging conflicts enables tracking recurring inconsistencies, systematically tests rather than subjectively judged.

Arbitration pipeline:

```
[EVENT_A] [EVENT_B]
  |
  v
  [Conflict Detector]
    |  compare shared fields (topic, scope, time_window)
    |  if overlapping and different → conflict
    |
    v
  [Arbitration Agent]
    |  LLM receives both events + conflict description
    |  decides: which is more relevant to the query context
    |  or whether both are valid (different aspects)
    |
    v
  [Annotated Output]
    |  both events + arbitration_result + winner_index
    |  downstream consumers see the resolved view
    |  conflicts logged for offline analysis
```

Regress in code. the utility classifier's alternation mechanism serves as a natural contentment layer — it stops when the model produces a response without requesting further tools. Multi-step queries (e.g. "Find the Kafka topic and explain its retention") may require two iterations, where the wrapper injects the tool result between iterations.

Mitigations:

M1 — Replace LLM arbitration with a trained lightweight classifier as primary. Per Ferrari et al. (coreset), a few-hundred labelled events are sufficient to fine-tune a BERT-class model that outperforms an LLM classifier for structured field classification. The LLM classifier is retained as a fallback for novel conflict patterns

Figure 3 — Permutation trait evaluation

Mon Tue Wed Thu Fri Sat Sun Mon Tue Wed Thu Fri Sat Sun Mon Tue Wed Thu Fri Sat Sun

	Mon	Tue	Wed	Thu	Fri	Sat	Sun
Permutation	**+0.438**		**−1.00**			**+0.494**	
Random baseline							
Gap	**+0.494**		**+1.00**			**+0.494**	

Distributing rotations over clinics and caravans. 5 gets from cycle 1 (+1.0 and 0.0) or 4 gets from cycle 2 (−1.0 and 0.0). The gap is constant — early adolescent fans fan longest under both rotations. Permutation lifts Mon–Wed by +0.494 and Sat–Sun by +0.494 against random. Random baseline is 0.00. Permutation helps in cycle 1 (attention-rich days), not in cycle 2 (attention-poor days).

DPO pairs generated. The contradiction discover produced 1,000 pairs from the seeded contradictions — one per discover turn where the first claim appeared. Pairs are weighted at 0.1 (field penalty multiplier) and ready for calibration pipeline ingestion.

8.4 Permutation Evolution

Generation	Accuracy	Affect CI	Affect SI
Random baseline	0.000	0.000	0.000
Hand-designed	0.438	0.000	0.000
Evolved (gen 1)	0.494	0.000	0.000
Evolved (gen 2)	0.494	0.000	0.000
Evolved (gen 3)	0.494	0.000	0.000
Evolved (gen 4)	0.494	0.000	0.000
Evolved (gen 5)	0.494	0.000	0.000

Curiosity and novelty are stable through cycle 10 (contradiction can dampen the improving utility signal), then grow to ratio 1 and 2 as utility fitness learns positive and contradiction rate falls to zero. Affect CI and SI remain at 0.000 throughout — the fitness function does not reward affect, and evolution preserves the curiosity/novelty profile. Random rotation underperforms because the contradiction rate (1.0 → 0.25) shaped fitness like a threshold respondent trigger a learner would. Trials that were stable earlier in the drift run again and fixed-keyword rules are working—no change to applied-fact rule's measured data.

8.5 Simulation Gaps and Resolutions (v.s.)

Two concrete issues surfaced from the simulation and are not resolved for the Phase 1 live implementation:

1. Affect accumulation via EMA. The simulation computed affect per interaction against a fixed baseline, creating accumulation. The live implementation uses exponential moving average (EMA) with $\alpha = 0.3$. This means the system is straightforward — maintain a running domain level offset from rating the same. Both patterns are confidence:

```
# Simulation: fixed baseline, accumulation
affect_delta = rating - baseline
cumulative_affect += affect_delta

# Implementation: EMA decay, current interaction dominant
affect = 0.7 * affect + 0.3 * affect_delta
# Recent interactions weight more than accumulated history
```

So the system drifts with contexts naturally—influenced responses gradually move better against the user's actual pattern. Whether this is the right decay rate for long-term companionship is an open question with profitable field access. Setting this implementation bias for Phase 1 replays and internalizes affect with a fifth accumulated domain state persisted in the sacrificial store alongside confidence.

2. Semantic profile difficulty resolution. The simulation found difficulty profiles based the novelty-creatives never reset after cycle 1, collapsing curiosity to zero for cycles 2 and 3. This is a known design gap and not a prioritized fix. Phase 1's harvesters (Phase 2) implements difficulty resolution using cosine(claim) vs difficulty-form:

```
difficulty_resolved = (
    difficulty_vector · claim_vector
    > resolution_threshold
    AND
    cosine(difficulty_vector, claim_vector)
    > cosine(difficulty_vector, resolution_vector)
)

Novelty creative reward:
    novelty_score × (1 - difficulty_resolved)
    # Novelty matters only when difficulty persists
    # Resolved difficulty → novelty reward → 0
```

This ensures that as prior domains confirmed-true, harvest profiles are introduced, the novelty creative resets, and the curiosity growth function re-engages. The split-cap and gap bonus mechanisms run then for pragmatic-numerical pairs calibration (v.0).

Outcome: Both gaps are implementation-level, not architecture-level. The simulation surfaced them before live deployment. The EMA decay rate and semantic difficulty resolution are tunable parameters; the simulation's results tell them. The live system will exercise the full curiosity and affect dynamics that the simulation could not.

Appendix B: Mathematical Foundations

This appendix contains formal mathematical foundations for the utility function and system-level properties described in §3. Proofs are sketched where they are straightforward; the main text provides the intuitive and empirical justification. Notation follows the conventions of §3.

Notation. $\mathcal{D}$ = the set of all possible domains (|$\mathcal{D}$| = 16 in our implementation). $\mathcal{H}$ = Gibbs-Cantelli deviation (**B.1**). Field weight profiles — user-preferentially-weighted priorities (**Proposition B.1**). $\mathcal{E}$ = Effect-signed $\mathcal{H}$ = Bonus Williams interpretation (**Theorem B.1**). $\mathcal{H}$ = EMA confidence update — Bayesian-like belief update (**B.2**). $\mathcal{H}$ = Permutation — Mon–Wed vs Sat–Sun temporal-attention gap (**Proposition B.2**). $\mathcal{H}$ = Confidence convergence — quantitative convergence rate (**Proposition B.3**). $\mathcal{H}$ = Curiosity stability — Lyapunov analysis, bounded stable equilibrium.

Before analyzing the properties of the utility function U, we justify its structure from first principles. In current form,

$$U(\mathcal{H}, \mathcal{E}, \mathcal{H}) = w_1 \mathcal{H} + w_2 \mathcal{E} + w_3 \mathcal{H} + w_4 \mathcal{H}$$

may appear as a convenient aggregation. This section establishes that its structure is **not arbitrary**. Four axioms remarkably force all deviations on four preferences, consistency, and independent domain contribution to decision making.

B.1. Additive Linear Structure from Separability and Scaling

We seek a utility function $U : \mathbb{R}^4 \to \mathbb{R}$ over three measurable dimensions — effect $\mathcal{E}$, confidence $\mathcal{H}$, curiosity $\mathcal{H}$, — satisfying the following axioms.

Axiom (Monotonicity). U is strictly increasing in each argument.

Axiom (Continuity). U is continuous in $(\mathcal{E}, \mathcal{H}, \mathcal{H})$.

Axiom (Independence / Separability). The marginal effect of improving one dimension does not depend on the current level of the others. Formally, for any $\mathcal{E} \neq \mathcal{E}'$,

$$U(\mathcal{E}, \mathcal{H}, \mathcal{H}) - U(\mathcal{E}', \mathcal{H}, \mathcal{H}) = U(\mathcal{E}, \mathcal{H}', \mathcal{H}') - U(\mathcal{E}', \mathcal{H}', \mathcal{H}')$$

Axiom (Field-Invariance Structure). The functional form of U is invariant across fields; only the weights w_i^f change across fields f.

(i) Linear Scaling Invariance. For all $a > 0$, b and $(\mathcal{E}, \mathcal{H}, \mathcal{H}) \in \mathbb{R}^3$, $U(\mathcal{E}, \mathcal{H}, \mathcal{H}) = U(a\mathcal{E}, a\mathcal{H}, a\mathcal{H}) + b$.

Motivation. Any choice that scaling all dimensions by the same factor scale utility proportionally. This is natural when $\mathcal{E}$, $\mathcal{H}$, and $\mathcal{H}$ are all measured on the same interpretable $[0, 1]$ scale, so agent or field comparisons remain meaningful. Without this axiom, we would need to specify how to trade off effect against confidence differently for each field, introducing arbitrary cross-dimensional scaling parameters. The invariance requirement ensures that the same relative preferences hold across all fields.

Theorem B.1.

Under axioms (i)–(iv), the utility function is necessarily of the form

$$U(\mathcal{E}, \mathcal{H}, \mathcal{H}) = w_1(\mathcal{E} - \alpha_1) + w_2(\mathcal{H} - \alpha_2) + w_3(\mathcal{H} - \alpha_3)$$

with $w_1, w_2, w_3 > 0$ and $w_1 + 10w_2 + w_3 = 1$ (normalization).

Proof.

Step 1 — Additive representation from (ii–iv).

Axiom (iv) ensures that each component is preferentially independent of the others. Applying this preference $\mathcal{E}$ = Effect-signed $\mathcal{H}$ = Bonus Williams theorem (Theorem 1 in Keeney & Raiffa, 1976), we obtain mutual preferential independence. By the theorem of Gorman (1968), preferences satisfying this condition admit a cardinal representation with mutually preferentially independent components admits an additively separable representation. Therefore there exist continuous utility functions $u_1(\mathcal{E}), u_2(\mathcal{H}), u_3(\mathcal{H})$ such that

$$U(\mathcal{E}) = u_1(\mathcal{E}) + u_2(\mathcal{H}) + u_3(\mathcal{H})$$

Step 2 — Linearity from (i).

Substitute the additive form into Axiom (i):

$$u_1(a\mathcal{E}) + u_2(a\mathcal{H}) + u_3(a\mathcal{H}) = u_1(\mathcal{E}) + u_2(\mathcal{H}) + u_3(\mathcal{H}) + b$$

Put $\mathcal{E} = \mathcal{H} = \mathcal{H} = 0$ and let $A = u_1(0)$, $B = u_2(0)$, $C = u_3(0)$. Then

$$u_1(a\mathcal{E}) + u_2(a\mathcal{H}) + u_3(a\mathcal{H}) = u_1(\mathcal{E}) + u_2(\mathcal{H}) + u_3(\mathcal{H}) + b$$

The triple-knot-side dependent-only-on-a $\mathcal{H}$, $\mathcal{H}$, and $\mathcal{H}$ — the Pexiderized Cauchy equation. Its continuous solution is

$$u_1(\mathcal{E}) = \alpha_1 \mathcal{E} + A, \quad u_2(\mathcal{H}) = \alpha_2 \mathcal{H} + B, \quad u_3(\mathcal{H}) = \alpha_3 \mathcal{H} + C$$

where $\alpha_i \geq 0$ is a function of a alone. Differentiating with respect to a:

$$\mathcal{E}\alpha_1'(a\mathcal{E}) + \mathcal{H}\alpha_2'(a\mathcal{H}) + \mathcal{H}\alpha_3'(a\mathcal{H}) = \mathcal{E}\alpha_1'(\mathcal{E}) + \mathcal{H}\alpha_2'(\mathcal{H}) + \mathcal{H}\alpha_3'(\mathcal{H})$$

Setting $a = 1$ and denoting $\alpha_1' = \alpha_2' = \alpha_3' = \alpha'$, we get

$$\mathcal{E}\alpha_1'(\mathcal{E}) + \mathcal{H}\alpha_2'(\mathcal{H}) + \mathcal{H}\alpha_3'(\mathcal{H}) = \alpha'(\mathcal{E} + \mathcal{H} + \mathcal{H})$$

This holds for all $(\mathcal{E}, \mathcal{H}, \mathcal{H})$ in a neighborhood of $(0, 0, 0)$, so $\alpha_i'(x) = \alpha_i'(0) + \alpha_i' x$. Integrating:

$$\alpha_1(\mathcal{E}) = \alpha_1 \mathcal{E} + \alpha_1'\mathcal{E}^2/2, \quad \alpha_2(\mathcal{H}) = \alpha_2 \math

1. **Monotonicity.** C^* is strictly increasing in δ:

$$\frac{\partial C^*}{\partial \delta} = 1 - \lambda \rho(f) > 0$$

Higher agent pass rates produce higher steady-state confidence, all else equal.

1. **Field sensitivity.** C^* is strictly decreasing in $\rho(f)$:

$$\frac{\partial C^*}{\partial \rho(f)} = -\delta\lambda < 0$$

Higher-stakes fields (larger $\rho(f)$) impose a lower achievable steady-state confidence, correctly encoding that high-stakes domains demand stricter internal consistency standards.

1. **Contradiction recovery time.** Suppose the agent is at steady-state C^* and a contradiction event causes an instantaneous drop of magnitude δ, so $C_0 = C^* - \delta$. The number of subsequent interactions required to return to within ϵ of C^* is:

$$t_{\text{recovery}} = \left\lceil \frac{\log(\epsilon/\delta)}{\log(1-\alpha)} \right\rceil$$

Proof.

Part 1 — Existence and uniqueness. The fixed point satisfies $C^* = (1-\alpha)C^* + \alpha\delta^*$, giving $\alpha C^* = \alpha\delta^*$, hence $C^* = \delta^* = \delta(1 - \lambda\rho(f))$. This is unique since the equation is linear in C^*.

Part 2 — Geometric convergence in expectation. Define the error $e_t = C_t - C^*$. Subtracting the fixed-point equation from the update rule:

$$e_{t+1} = (1-\alpha)e_t + \alpha(\delta_t - \delta^*)$$

The noise term $\eta_t \triangleq \alpha(\delta_t - \delta^*)$ has zero mean under the stationary distribution (since $\mathbb{E}[\delta_t] = \delta^*$) but does not vanish pathwise. Taking expectations:

$$\mathbb{E}[e_{t+1}] = (1-\alpha)\,\mathbb{E}[e_t] + \alpha\,\mathbb{E}[\delta_t - \delta^*] = (1-\alpha)\,\mathbb{E}[e_t]$$

Iterating from $t = 0$ with $e_0 = C_0 - C^*$ deterministic:

$$\mathbb{E}[e_t] = (1-\alpha)^t\,(C_0 - C^*)$$

By Jensen's inequality $|\mathbb{E}[e_t]| \leq \mathbb{E}[|e_t|]$, and applying the triangle inequality to the closed-form solution:

$$\mathbb{E}[|e_t|] \leq (1-\alpha)^t\,|C_0 - C^*| + \alpha\sum_{k=0}^{t-1}(1-\alpha)^{t-1-k}\,\mathbb{E}[|\delta_k - \delta^*|]$$

The second term represents the expected accumulated noise. Under the stationary assumption with zero-mean noise, $\mathbb{E}[\delta_k - \delta^*]$ is a constant σ_ϵ and the noise sum telescopes to $\alpha\sigma_\epsilon/(\alpha) = \sigma_\epsilon$. This gives the tighter statement:

$$\mathbb{E}[|e_t|] \leq (1-\alpha)^t\,|C_0 - C^*| + \sigma_\epsilon$$

where $\sigma_\epsilon = \mathbb{E}[|\delta_t - \delta^*|]$ is the mean absolute deviation of the effective signal. When the signal has low noise ($\sigma_\epsilon \approx 0$), the bound reduces to the clean geometric form. **The bound** $\mathbb{E}[|e_t|] \leq (1-\alpha)^t|C_0 - C^*|$ **holds exactly when** $\sigma_\epsilon = 0$ **(deterministic signal) and approximately when noise is small.**

The half-life of the mean error is $t_{1/2} = \log(1/2)/\log(1-\alpha)$. For $\alpha = 0.2$: $t_{1/2} = \log(0.5)/\log(0.8) = 3.11$, so $\lceil t_{1/2} \rceil = 13$ interactions.

Part 3 — Monotonicity. $\partial C^*/\partial\delta = (1 - \lambda\rho(f))$. Since $\lambda \in [0, 1]$ and $\rho(f) \geq 1$ but $\lambda\rho(f) < 1$ for any non-degenerate field (confidence cannot be driven to zero by a single contradiction), this derivative is strictly positive.

Part 4 — Field sensitivity. $\partial C^*/\partial\rho(f) = -\delta\lambda \leq 0$, with strict inequality whenever $\delta > 0$ and $\lambda > 0$. This formalizes the intuition that high-stakes fields (large $\rho(f)$) penalize contradictions more heavily, pulling the achievable steady-state confidence downward even when pass rates are high.

Part 5 — Recovery time. After the contradiction, $e_t = -\delta$ (deterministic drop). In the noise-free case ($\sigma_\epsilon = 0$), by geometric convergence, $|e_{t+k}| \leq (1-\alpha)^k\,\delta$. We want this below ϵ:

$$(1-\alpha)^k\,\delta \leq \epsilon \implies k \geq \frac{\log(\epsilon/\delta)}{\log(1-\alpha)}$$

Since $\log(1-\alpha) < 0$ and $\epsilon < \delta$ (we want to recover closer than the drop), $\epsilon/\delta < 1$ and $\log(\epsilon/\delta) < 0$, making t positive. Therefore:

$$t_{\text{recovery}} = \left\lceil \frac{\log(\epsilon/\delta)}{\log(1-\alpha)} \right\rceil \qquad \blacksquare$$

Worked Examples

Example 1 — Software engineering, no contradictions.

$\mu(f) = 2$, $\lambda = 0$, $\delta = 0.85$, $C_0 = 0.5$, $\alpha = 0.2$.

$$C^* = 0.85 \times (1 - 0 \times 2) = 0.85$$

$$|C_0 - 0.85| \leq 0.8^t \times 0.35$$

After 10 interactions: $|e_{10}| \leq 0.8^{10} \times 0.35 = 0.107 > 0.035 = 0.027$. After 20 interactions: $|e_{20}| \leq 0.8^{20} \times 0.35 = 0.0038$.

Example 2 — Surgery, boundary case.

$\mu(f) = 10$, $\lambda = 0.1$ (moderate per-interaction penalty), $\delta = 0.90$.

$$C^* = 0.90 \times (1 - 0.1 \times 10) = 0.90 \times 0 = 0$$

This represents an extreme boundary case; in practice λ is small per interaction (contradictions are rare events, not a sustained rate). The example illustrates correct model behavior at the boundary: when $\lambda\rho(f) \approx 1$, the contradiction penalty exactly cancels the pass-rate signal, and the agent cannot build confidence regardless of performance. This is the intended behavior — a surgical agent that consistently contradicts itself on verified claims should be unable to achieve operating confidence and must abstain.

Example 3 — Recovery time.

Starting from a drop of $\delta = 0.15$ (a significant contradiction), recovering to within $\epsilon = 0.01$ of C^* with $\alpha = 0.2$:

$$t_{\text{recovery}} = \left\lceil \frac{\log(0.01/0.15)}{\log(0.8)} \right\rceil = \left\lceil \frac{-2.708}{-0.223} \right\rceil = \lceil 12.1 \rceil = 13 \text{ interactions}$$

Thirteen clean calibration interactions are sufficient to recover from a large contradiction event to within 1% of steady-state confidence. This is consistent with the simulation results in Appendix A.

Corollary — Steady-State Confidence Bounds by Field

Substituting typical field parameters with $\delta = 0.85$ (a well-calibrated agent) and $\lambda = 0.05$ (low contradiction rate):

Field	$\rho(f)$	$C^* = \delta(1 - \lambda\rho)$	$C_{\min}$	Achievable?
Surgery / Aviation	10	$0.85 \times 0.5 = 0.425$	0.95	No — requires $\delta > 0.95/(1 - 0.5) = 1.9$
Law	5	$0.85 \times 0.75 = 0.638$	0.85	Borderline
Software Engineering	2	$0.85 \times 0.9 = 0.765$	0.70	Yes
Creative Writing	1	$0.85 \times 0.95 = 0.808$	0.55	Easily

This corollary formalizes the whitepaper's claim that high-stakes fields impose stricter confidence standards: for surgery, a typical agent with $\delta = 0.85$ and any nonzero contradiction rate cannot achieve the $C_{\min} = 0.95$ threshold, and must abstain or escalate. The confidence floor is not merely a policy choice — it is above the achievable steady-state, guaranteeing the abstention mechanism is triggered when the agent is not reliably correct.

B.7 — Theorem B.7 — Lyapunov Stability of the Personality System

Corrected Claim

The personality system exhibits **bounded, stable dynamics with convergence to a neighborhood of the field neutral s^*, under bounded drift.** We do not claim a unique globally stable equilibrium, which would require the mean reversion to dominate the drift — a condition that does not hold for the parameter values $\beta = 0.01$, $\Delta_{\max} = 0.05$ used in this system. Instead we prove the three things that are true: invariance of the feasible set, geometric convergence to s^* when drift is absent, and bounded dynamics otherwise.

Setup

The personality trait vector $s = (s_1, \ldots, s_n) \in \mathbb{R}^n$ *evolves* under:

$$s_{t+1} = \Pi_B(s + \Delta_t - \beta(s_t - s^*)) = \Pi_B[(1 - \beta)s_t + \beta s^* + \Delta_t]$$

where:

- $B = \prod_{i=1}^n [\underline{s}_{\max}, \bar{s}_{\max}]$ is the field-specific feasible set (a closed convex box in $\mathbb{R}^n$)
- $\Pi_B : \mathbb{R}^n \to B$ is the Euclidean projection onto B
- $\Delta_t \in \mathbb{R}^n$ is the raw drift from utility history, with $\|\Delta_t\|_\infty \leq \Delta_{\max}$ per component
- $\beta = 0.01$ is the mean reversion coefficient
- $s^* \in B$ is the field neutral personality (interior point of B by construction)

Define the Lyapunov function:

$$V(s) = \|s - s^*\|^2 = \sum_{k=1}^n (s_k - s_k^*)^2 \geq 0$$

with $V(s^*) = 0$.

B.7 — Theorem B.7 — Bounded Stable Dynamics with Neighborhood Convergence

Theorem. *Under the three-layer personality evolution rule above:*

(i) Invariance. $s_t \in B$ for all $t \geq 0$ whenever $s_0 \in B$.

(ii) Zero-drift convergence. When $\Delta_t = 0$ for all t, $V(s_t)$ converges geometrically to zero:

$$V(s_{t+1}) \leq (1 - \beta)^2\,V(s_t)$$

so $\|s_t - s^*\| \leq (1 - \beta)^t\|s_0 - s^*\|$, with *convergence rate* $(1 - \beta)^2 = 0.9801$ *per cycle.*

(iii) Bounded displacement. The single-step displacement is bounded:

$$\|s_{t+1} - s_t\| \leq \Delta_{\max}\sqrt{n} + \beta\,\|s_t - s^*\|$$

so no single evolution cycle can produce a large jump.

(iv) Persistent-drift stability. Under persistent drift with $\|\Delta_t\| \leq \Delta_{\max}\sqrt{n}$, the distance to s^* satisfies:

$$\|s_{t+1} - s^*\| \leq (1 - \beta)\|s_t - s^*\| + \Delta_{\max}\sqrt{n}$$

The dynamics converge to and remain in the neighborhood

$$\mathcal{N}^* = \{s \in B : \|s - s^*\| \leq r^*\}, \qquad r^* = \min\left(\frac{\Delta_{\max}\sqrt{n}}{\beta},\ \operatorname{diam}(B)\right)$$

For the system parameters $\beta = 0.01$, $\Delta_{\max} = 0.05$, $n = 6$ (traits), $\Delta_{\max}\sqrt{n}/\beta = 12.25$, while $\operatorname{diam}(B) \approx \sqrt{n} \approx 2.45$. The field bounds are therefore the binding constraint, not the mean reversion — $\mathcal{N}^* = B$.

Proof

Part (i) — Invariance.

Π_B is defined as the Euclidean projection onto the closed convex set B, so $\Pi_B(x) \in B$ for every $x \in \mathbb{R}^n$. Therefore $s_{t+1} = \Pi_B(\cdot) \in B$ for all $t \geq 0$, regardless of Δ_t. $\blacksquare$

Part (ii) — Zero-drift convergence.

Set $\Delta_t = 0$. Then $s_{t+1} = \Pi_B[(1 - \beta)s_t + \beta s^*]$.

Since B is convex and both $s_t \in B$ and $s^* \in B$, the convex combination $(1 - \beta)s_t + \beta s^* \in B$, so Π_B acts as the identity:

$$s_{t+1} = (1 - \beta)s_t + \beta s^*$$

Therefore:

$$s_{t+1} - s^* = (1 - \beta)s_t + \beta s^* - s^* = (1 - \beta)(s_t - s^*)$$

and:

$$V(s_{t+1}) = \|(1 - \beta)(s_t - s^*)\|^2 = (1 - \beta)^2\,V(s_t). \qquad \blacksquare$$

The convergence rate per cycle is $(1 - \beta)^2 = (0.99)^2 = 0.9801$. The half-life is:

$$t_{1/2} = \frac{\log(1/2)}{\log(1 - \beta)^2} = \frac{\log(1/2)}{2\log(0.99)} = \frac{-0.693}{-0.0201} \approx 34 \text{ cycles}$$

With personality evolution running every $N = 3$ interactions, this corresponds to approximately 102 interactions to halve the distance to s^* under zero drift.

Part (iii) — Bounded displacement.

$$\|s_{t+1} - s_t\| = \|\Pi_B[(1 - \beta)s_t + \beta s^* + \Delta_t] - s_t\|$$

Since Π_B is non-expansive and $s_t = \Pi_B(s_t)$:

$$\begin{aligned}
\|\Pi_B(x) - \Pi_B(y)\| &\leq \|x - y\| \quad \text{for all } x, y \\
\|s_{t+1} - s_t\| &\leq \|(1 - \beta)s_t + \beta s^* + \Delta_t - s_t\| \\
&= \|-\beta(s_t - s^*) + \Delta_t\| \leq \beta\,\|s_t - s^*\| + \|\Delta_t\| \leq \beta\,\|s_t - s^*\| + \Delta_{\max}\sqrt{n} \qquad \blacksquare
\end{aligned}$$

Part (iv) — Persistent-drift stability.

Since $s^* \in B$, the non-expansiveness of Π_B with respect to any point in B gives:

$$\begin{aligned}
\|s_{t+1} - s^*\| &= \|\Pi_B[(1 - \beta)s_t + \Delta_t] - \Pi_B(s^*)\| \\
&\leq \|(1 - \beta)s_t + \beta s^* + \Delta_t - s^*\| \\
&= \|(1 - \beta)(s_t - s^*) + \Delta_t\| \\
&\leq (1 - \beta)\|s_t - s^*\| + \|\Delta_t\| \\
&\leq (1 - \beta)\|s_t - s^*\| + \Delta_{\max}\sqrt{n}
\end{aligned}$$

Let $d_t = \|s_t - s^*\|$. The recurrence $d_{t+1} \leq (1 - \beta)d_t + \Delta_{\max}\sqrt{n}$ has fixed point $d^* = \Delta_{\max}\sqrt{n}/\beta$. By the theory of contractive linear recurrences:

- If $d_t > d^*$: $d_{t+1} < d_t$ (distance decreasing toward d^*)
- If $d_t \leq d^*$: $d_{t+1} \leq d^*$ (distance stays within neighborhood)

So $\limsup_{t\to\infty} d_t \leq d^* = \Delta_{\max}\sqrt{n}/\beta$.

Since $d_t \leq \operatorname{diam}(B)$ always by Part (i), the binding constraint is:

$$r^* = \min\left(\frac{\Delta_{\max}\sqrt{n}}{\beta},\ \operatorname{diam}(B)\right) \qquad \blacksquare$$

Why the Field Bounds Are the Binding Constraint

For the system parameters:

$$\frac{\Delta_{\max}\sqrt{n}}{\beta} = \frac{0.05 \times \sqrt{6}}{0.01} = \frac{0.1225}{0.01} = 12.25$$

$$\operatorname{diam}(B) = \|s_{\max} - s_{\min}\| \leq \sqrt{n} \approx 2.45 \quad \text{(since each trait is in } [0, 1])$$

Since $12.25 \gg 2.45$, the mean reversion alone is insufficient to confine the dynamics to a small neighborhood of s^*. **The projection Π_B is the primary stability mechanism** — it enforces invariance, and the field bounds define how far from s^* the personality can stray. The mean reversion serves as a regularizer: without it, the system would drift to and remain at the boundary of B; with it, there is a gentle restoring force toward s^* when drift is absent.

This clarifies the design role of each stability layer:

Layer	Mechanism	Guarantee
Field bounds $[\underline{s}_{\max}, \bar{s}_{\max}]$	Projection Π_B	Hard invariance: $s_t \in B$ always
Drift rate cap $\Delta_{\max}$	Truncation of raw drift	Bounded single-step displacement
Mean reversion β s^*	Soft pull toward	Convergence to s^* when drift is absent; regularization otherwise

---

Corollary — No Oscillation

Corollary. *The drift rate cap $\Delta_{\max}$ prevents oscillation: after any evolution step, the personality cannot cross s^* in a single step.*

Proof. A step crosses s^* in dimension i if $s_{t+1}^i - s_t^i$ and $s_t^i - s_t^*$ have opposite signs. The signed displacement in dimension i is:

$$s_{t+1}^i - s_t^i = \Pi_{B,i}(\cdot) - s_t^i = [(1 - \beta)(s_t^i - s_t^*) + \Delta_t^i]$$

For oscillation, we need $|\Delta_t^i| > (1 - \beta)|s_t^i - s_t^*|$, i.e., the drift must exceed the current displacement. Since $\Delta_t^i \leq \Delta_{\max} = 0.05$ and $s_t^i - s_t^*$ can be as large as $s_{\max}^i - s_t^*$ (which is at least 0.05 by the bound structure), oscillation requires the personality to be within $\Delta_{\max}/(1 - \beta) = 0.05/0.99 \approx 0.05$ of s^* in that dimension. This is a vanishingly small region, confirming that oscillation can only occur when the personality is already very near the neutral point. $\blacksquare$

Remark on Parameter Calibration

The analysis reveals a structural tension: mean reversion $\beta = 0.01$ is weaker than the maximum drift $\Delta_{\max} = 0.05$ per cycle. This is intentional — personality is meant to evolve meaningfully, not snap back to s^* every few cycles. The design trades off:

- **Large** β: fast convergence to s^*, but personality becomes unresponsive to experience
- **Small** β: personality evolves with experience, but reverts slowly between periods of drift

The value $\beta = 0.01$ produces a half-life of approximately 34 evolution cycles under zero drift (~102 interactions), which is long enough for the personality to reflect accumulated experience over hundreds of interactions before the neutral pull becomes dominant. A field-specific $\beta(f)$ — with higher reversion rates for high-stakes fields where personality stability is more important — is a natural extension for future work.

References

- Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine Learning, 47*(2–3), 235–256.
- Blackwelder, C., & Donaldson, D. (1981). Ratio-scale and translation-scale full interpersonal comparability without domain restrictions. *International Economic Review, 22*(2), 249–268.
- Debreu, G. (1960). Topological methods in cardinal utility theory. In K. J. Arrow et al. (Eds.), *Mathematical Methods in the Social Sciences*. Stanford University Press.
- Kalman, R. E. (1960). A new approach to linear filtering and prediction problems. *Journal of Basic Engineering, 82*(1), 35–45.
- Mann, H. B., & Whitney, D. R. (1947). On a test of whether one of two random variables is stochastically larger than the other. *Annals of Mathematical Statistics, 18*(1), 50–60.
- Wald, A. (1945). Sequential tests of statistical hypotheses. *Annals of Mathematical Statistics, 16*(2), 117–186.